설비 고장을 예측하는 AI 데이터 분석

Pandas 데이터 선택·필터링

행·열 선택

posco

CONTENTS

목차

01 열 골라내기 — 대괄호 한 겹과 두 겹

02 loc 과 iloc — 라벨과 위치, 슬라이싱 함정

03 종합 실습 — 필요한 구간만 뽑아내기

posco

이 단원의 실데이터 — 주조 다이캐스팅

행·열 선택을 연습할 데이터가 어떤 설비에서 나왔는지 먼저 이해

공정 — 주조 다이캐스팅

용융 금속을 금형에 고압으로 주입해 부품을 성형하는 설비. 한 번의 주입(샷)마다 압력·두께·형체력을 기록한다.

핵심 변수와 물리적 의미

설비ID(DIE-A-01 형식의 호기 이름) · 실린더압력·주조압력(주입 압력) · 사이클타임(1샷 소요 시간) · 비스킷두께(잔류 금속 두께) · 형체력(금형을 닫는 힘) · 품질등급(양품/주의/불량).

정상과 이상

압력·형체력이 정상 범위이고 비스킷두께가 적정하면 양품. 형체력 부족·비스킷 이상·사이클 지연이면 주의, 실제 결함이면 불량으로 기록된다.

왜 행·열 선택인가

수백 샷 로그에서 원하는 샷(행)과 지표(열)만 정확히 골라내야 품질을 빠르게 분석할 수 있다. 열 이름으로 고르는 길과 순서 번호로 고르는 길 두 가지를 익힌다.

01

열 골라내기

대괄호 한 겹과 두 겹 — 무엇이 Series이고 무엇이 표인가

posco

번호로 하던 것을 이름으로도

앞 단원의 배열 인덱싱과 하는 일은 같다 — 새로 생기는 것은 이름으로 고르는 길

비교 항목	NumPy 배열 (앞 단원)	Pandas 표 (이 단원)
무엇을 고르나	`arr[:, 3]`	`df['형체력']`
여러 개 고르기	`arr[:, [3, 1]]`	`df[['형체력', '실린더압력']]`
고르는 기준	열 번호만	**열 이름 또는 열 번호**
3번 열이 무엇인지	매번 세어 봐야 함	**이름이 그대로 적혀 있음**

하는 일은 똑같다. 번호로 세는 대신 이름으로 부를 수 있게 된 것이 전부다.

이름으로 부를 수 있으니 코드를 나중에 다시 읽어도 무엇을 골랐는지 바로 보인다

posco

질문이 곧 열 선택으로 이어진다

무엇이 궁금한지를 정하면 어떤 열을 볼지가 따라옴

01 · 미성형 점검

금형이 덜 닫혔나
→ 형체력 열

형체력이 떨어지면 금속이 새어 미성형 — 형체력 열만 뽑아 추세 확인

02 · 사이클 지연 점검

한 샷이 오래 걸리나
→ 사이클타임 열

사이클타임이 길어지면 공정 지연 의심 — 사이클타임 열만 뽑아 확인

질문이 흐릿하면 어떤 열을 볼지도 흐릿해짐 — 좋은 분석은 명확한 질문에서 출발

단일 열 선택

데이터프레임 뒤에 대괄호를 붙이고 열 이름을 따옴표로 감싸기

```
df['형체력']
```

위 식은 **형체력 열의 모든 값**을 세로로 출력

주의 — 따옴표 빠짐

따옴표를 빼면 파이썬이 형체력을 변수 이름으로 찾다가 오류 발생
띄어쓰기·대소문자가 한 글자라도 다르면 **"그런 열은 없다"** 오류

posco

단일 열 선택 — 코드

대괄호 + 따옴표로 열 하나 꺼내기

CODE

```
df = pd.read_csv('data/13_diecasting_small.csv')
형체력값 = df['형체력']
print(형체력값.head(3))
# 0    258.0
# 1    257.0
# 2    253.0
# Name: 형체력, dtype: float64
```

값 왼쪽에 인덱스가 함께 붙어 어느 샷의 형체력인지 추적 가능 · 큰따옴표도 결과 동일

posco

복수 열 선택

여러 열은 대괄호 안에 리스트로 묶어 선택 — 대괄호 두 겹

```
df[['형체력', '실린더압력']]
```

대괄호 두 겹의 의미

바깥 [ ] — 골라내기 신호
안쪽 [ ] — 리스트로 묶었다는 표시
적은 순서대로 결과 표의 열 순서 결정

자주 하는 실수

안쪽 대괄호 빠뜨림
대괄호 하나로 여러 이름을 쓰면 오류
여러 열은 **반드시 두 겹**

복수 열 선택 — 코드

열 이름들을 리스트로 묶어 여러 열 선택

CODE

```
두열 = df[['형체력', '실린더압력']]
print(두열.shape)          # (30, 2)

순서변경 = df[['실린더압력', '형체력']]
print(list(순서변경.columns))  # ['실린더압력', '형체력']
```

결과는 DataFrame · 적은 순서가 그대로 열 순서 — 실린더압력을 먼저 적으면 앞 열

posco

대괄호 1개와 2개의 차이

같은 형체력 열도 대괄호 겹수에 따라 결과 형태가 달라짐

01 · 한 겹

df['형체력']

→ Series

한 줄짜리 1차원 데이터 — 바구니 없이 값만 꺼낸 것

02 · 두 겹

df[['형체력']]

→ DataFrame

열이 하나여도 표 형태 — 여러 열을 담을 수 있는 바구니

외우는 법 — 두 겹은 바구니, 하나만 들어 있어도 바구니 = DataFrame

posco

한 겹과 두 겹은 결과의 모양이 다르다

같은 형체력 열을 골라도 대괄호를 몇 겹 쓰느냐로 Series 가 되기도 표가 되기도 함

df['형체력'] 한 겹

0	258.0
1	257.0
2	253.0
3	254.0

Name: 형체력, dtype: float64

Series — 열 이름이 사라진 값의 줄

두 겹은 바구니

df[['형체력']] 두 겹

	형체력
0	258.0
1	257.0
2	253.0
3	254.0

DataFrame — 열 이름이 살아 있는 표

여러 열을 담을 수 있는 바구니에 열이 하나만 들어 있어도 바구니 — 두 겹은 언제나 DataFrame

한 겹은 값의 줄만 남고, 두 겹은 열 이름이 붙은 표가 그대로 남는다.

평균 하나만 필요하면 한 겹, 표를 계속 이어서 다룰 것이면 두 겹

posco

실습 1. 열 골라 꺼내기

대괄호 한 겹과 두 겹으로 각각 골라 결과의 모양을 직접 확인

목표

열 하나를 골랐을 때와 여러 개를 골랐을 때 결과의 모양이 어떻게 달라지는지 확인

단계

① 연습용 30줄짜리 다이캐스팅 파일을 읽어 변수에 담기

② 형체력 칸 하나만 대괄호 한 겹으로 골라 종류와 앞 세 값을 출력

③ 형체력과 실린더압력 두 칸을 대괄호 두 겹으로 골라 종류와 크기를 출력

④ 두 겹 안에 형체력 하나만 담아 보고 종류가 무엇으로 나오는지 확인

예상 결과

한 겹은 한 줄짜리 자료 · 두 겹은 칸이 하나여도 표 · 크기는 30줄 2칸

실습 1 결과 해석

세 가지만 확인하면 이 실습은 끝난다

원리

대괄호 겹수가 결과의 모양을 정한다

한 겹은 열 이름이 꼬리표로 밀려난 Series, 두 겹은 열 이름이 살아 있는 DataFrame.

고른 열이 무엇이냐가 아니라 어떻게 감쌌느냐가 모양을 정한다.

함정

두 겹인데 안쪽 대괄호를 빠뜨림

df['형체력', '실린더압력'] 처럼 쓰면 KeyError 가 난다.

여러 이름은 반드시 리스트로 묶어야 하므로 대괄호가 두 겹이 된다.

검산

shape 로 확인

두 겹 결과의 shape 가 (30, 2) 로 나오면 제대로 두 열을 담은 것이다.

한 겹 결과에는 shape 가 (30,) 으로 숫자가 하나뿐이다.

posco

실습 1 정답

대괄호 한 겹과 두 겹의 결과를 나란히 확인

CODE

```
import pandas as pd

df = pd.read_csv('data/13_diecasting_small.csv')

한겹 = df['형체력']
print(type(한겹))        # <class 'pandas.Series'>
print(한겹.head(3).tolist())   # [258.0, 257.0, 253.0]

두겹 = df[['형체력', '실린더압력']]
print(type(두겹))        # <class 'pandas.DataFrame'>
print(두겹.shape)        # (30, 2)

print(type(df[['형체력']]))   # 열이 하나여도 DataFrame
```

출력의 Series 와 DataFrame 을 한 줄씩 대조하며 읽을 것

posco

02

loc 과 iloc

라벨로 고르는 길과 위치로 고르는 길 — 그리고 슬라이싱 함정

posco

라벨과 위치 번호의 차이

오늘 모듈에서 가장 중요한 개념 — 라벨이냐 위치냐

01 · 라벨

그 행에 붙은
이름표 = 인덱스 값

인덱스 값 자체 — 0, 17, 양품처럼 데이터가 들고 있는 라벨

02 · 위치 번호

맨 위에서부터
몇 번째인지

표 안에서의 순서 — 항상 0부터 시작

인덱스가 0, 1, 2로 깔끔하면 둘이 똑같음 — 하지만 어긋나는 순간이 반드시 옴

라벨과 위치가 갈라지는 순간

정렬 한 번이면 이름표와 순서가 어긋난다 — 이때부터 loc 과 iloc 이 서로 다른 답을 준다

정렬 전 — 라벨 = 위치

라벨	위치	샷	형체력
0	0	1	258
1	1	2	257
2	2	3	253
3	3	4	254

sort_values →

형체력 내림차순 정렬 후

라벨	위치	샷	형체력
22	0	23	359
17	1	18	357
27	2	28	355
12	3	13	354

정렬 뒤 맨 위 행은 위치로는 0번, 라벨로는 22번 df.iloc[0] 은 샷 23, df.loc[0] 은 여전히 샷 1

정렬해도 이름표는 행을 따라 움직이고, 위치 번호는 자리에 남는다.

그래서 정렬한 표에서는 loc 과 iloc 이 서로 다른 행을 가리킨다 — 정렬 자체는 다음 시간에 자세히

posco

reset_index — 어긋난 번호 다시 붙이기

정렬한 표를 이어서 쓸 때 실무에서 거의 언제나 붙이는 한 줄

CODE

```
정렬 = df.sort_values('형체력', ascending=False)
print(정렬.index[:4].tolist())        # [22, 17, 27, 12]   이름표가 뒤죽박죽

재번호 = 정렬.reset_index(drop=True)
print(재번호.index[:4].tolist())      # [0, 1, 2, 3]
print(재번호.loc[0, '샷'], 재번호.iloc[0]['샷'])   # 23 23   다시 같아짐

# drop=True 를 빼면 옛 이름표가 index 라는 열로 남는다
print(정렬.reset_index().shape)       # (30, 9)   열이 하나 늘어남
```

정렬·필터 결과를 이어서 쓸 것이면 reset_index(drop=True) — 안 붙이면 loc 이 계속 옛 이름표를 본다

posco

loc과 iloc의 핵심 구분

오늘 모듈의 심장 — 이 두 글자 차이가 전부

DF.LOC[ ]

이름표
라벨로 선택

인덱스가 품질등급·날짜 같은 글자여도 사용 가능

DF.ILOC[ ]

순서
위치 번호로 선택

i = integer — 0부터 시작하는 정수만 받음

외우는 요령 — iloc의 i는 integer · loc은 글자 인덱스에도 가능

posco

loc과 iloc 비교표

네 가지 비교 항목 — 기준 · 받는 값 · 글자 인덱스 · 조건 필터

구분	loc	iloc
기준	**라벨 (이름표)**	**위치 (순서)**
받는 값	인덱스 값	0부터 시작하는 정수
글자 인덱스	**가능**	**불가**
조건 필터	**가능**	**불가**

조건 필터(예: 형체력 > 300) — 다음 단원에서 loc 과 함께 본격적으로 활용

posco

loc 단일 행 선택

df.loc 뒤 대괄호에 인덱스 라벨 작성

```
df.loc[0]
```

이름표가 0인 행 전체를 선택

중요 — 여기서 0의 의미

여기서 0은 **위치(첫 번째)가 아니라 이름표**
지금은 인덱스가 순서대로라 결과가 같아 보이지만 의미는 어디까지나 이름표

posco

loc 행 범위 선택

콜론으로 범위 — loc은 끝 라벨까지 포함

```
df.loc[0:4]
```

0, 1, 2, 3, 4 — 다섯 줄

loc만의 매우 중요한 특징

loc은 **끝에 적은 라벨까지 포함** — 문자열 슬라이싱과 정반대
잠시 후 배울 iloc은 끝을 포함하지 않아 결과가 다름

posco

iloc 위치 기반 선택

인덱스와 무관 — 위에서부터 세는 순서

```
df.iloc[0]
```

인덱스가 무엇이든 맨 위 첫 번째 행

iloc의 두 가지 제약

① **정수만 받음** — '양품' 같은 글자는 오류 · i = integer
② **조건 필터링 불가** — 순서 번호와 그 범위만

posco

실습 2. 라벨과 번호로 행 골라내기

정렬 전후로 loc 과 iloc 이 같은 행을 가리키는지 직접 확인

목표

줄을 세우기 전과 후에 이름표로 부른 행과 순서로 부른 행이 같은지 비교

단계

① 연습용 30줄짜리 다이캐스팅 파일을 읽어 변수에 담기

② 줄을 세우기 전에 이름표 0번 행과 맨 위 행의 샷 번호를 각각 출력

③ 형체력이 큰 순서로 줄을 세운 결과를 새 변수에 담기

④ 줄 세운 표에서 다시 두 방식으로 샷 번호를 뽑아 비교

예상 결과

줄 세우기 전에는 둘 다 샷 1 · 줄 세운 뒤에는 샷 1과 샷 23으로 갈림

실습 2 결과 해석

정렬 전후를 나란히 놓고 세 가지를 확인

원리

이름표는 따라가고 번호는 남는다

정렬은 행을 통째로 옮긴다. 라벨은 그 행에 붙어 함께 이동하고, 위치 번호는 자리에 그대로 남는다.

그래서 loc 과 iloc 이 갈라진다.

함정

정렬 뒤 loc[0] 을 맨 위로 착각

위험한 순으로 정렬해 놓고 loc[0] 으로 맨 위를 꺼냈다고 믿는 실수.

맨 위 한 줄이 필요하면 iloc[0] 이나 head(1) 을 써야 한다.

검산

샷 번호로 확인

정렬 후 iloc[0] 의 샷이 23,
loc[0] 의 샷이 1 로 나오면 맞다.

둘이 같게 나왔다면 정렬 결과가 아니라 원본을 보고 있는 것이다.

정렬한 표를 다룰 때는 지금 부르는 0이 이름표인지 순서인지 매번 되물을 것

posco

실습 2 정답

정렬 전후로 loc 과 iloc 을 나란히 비교

CODE

```
df = pd.read_csv('data/13_diecasting_small.csv')

print(df.loc[0, '샷'])       # 1
print(df.iloc[0]['샷'])      # 1   — 아직 둘이 같다

정렬 = df.sort_values('형체력', ascending=False)

print(정렬.loc[0, '샷'])     # 1    — 이름표 0번 행
print(정렬.iloc[0]['샷'])    # 23   — 맨 위 행
print(정렬.index[:3].tolist())   # [22, 17, 27]
```

마지막 줄의 인덱스 목록을 보면 왜 두 결과가 갈라지는지 한눈에 보인다

posco

loc 행과 열 동시 선택

loc의 진짜 힘 — 사각형 영역만 정확히 추출

```
df.loc[행,열]
```

쉼표 앞은 행, 쉼표 뒤는 열 — 행 먼저, 열 나중

외우는 법 — 줄, 즉 행이 먼저

표 읽을 때도 "몇 번째 줄의 어느 칸" — 순서 헷갈리면 엉뚱한 데이터

posco

행과 열을 한 번에 자르면 사각형이 남는다

쉼표 앞은 행, 뒤는 열 — 두 방향을 동시에 지정해 필요한 사각형만 추출

	샷	설비ID	실린더압력	주조압력	사이클타임	비스킷두께	형체력	품질등급
0	1	DIE-B-02	214.0	1037.0	20.7	10.0	258.0	양품
1	2	DIE-C-01	217.0	1052.0	20.7	11.0	257.0	양품
2	3	DIE-A-02	215.0	1040.0	20.7	21.0	253.0	주의
3	4	DIE-B-01	214.0	1037.0	20.8	11.0	254.0	양품
4	5	DIE-C-01	108.0	522.0	652.3	14.0	222.0	불량

df.loc[0:4, ['실린더압력', '형체력']] → (5, 2)

쉼표 앞은 행, 뒤는 열 — 두 방향을 한 번에 잘라 사각형만 남긴다

행 조건과 열 목록을 쉼표로 나눠 적으면 그 교차 영역만 남는다.

loc 행과 열 — 코드

쉼표 앞은 행, 뒤는 열 — 콜론 하나만 적으면 전체

CODE

```
부분 = df.loc[0:4, ['실린더압력', '형체력']]
print(부분.shape)     # (5, 2)

전체행 = df.loc[:, ['실린더압력', '형체력']]
print(전체행.shape)   # (30, 2)
```

loc 은 끝 라벨 포함 — 0:4 는 다섯 줄 · 행 자리에 콜론 하나만 적으면 모든 행

posco

iloc 행과 열 동시 선택

loc과 동일한 쉼표 구조 — 이름 대신 순서 번호

```
df.iloc[행번호 , 열번호]
```

열도 0부터 — 0번 샷, 1번 설비ID, 2번 실린더압력, 3번 주조압력, …

iloc 슬라이싱 — 끝 미포함

열 위치 2:4 → 2번과 3번(실린더압력·주조압력)
헷갈리면 columns로 열 순서 미리 확인

posco

iloc 행과 열 — 코드

행도 열도 순서 번호로 지정

CODE

```
print(df.iloc[0:2, 2:4])
# 행 0~1번, 열 2~3번
# = 실린더압력, 주조압력

print(list(df.columns))
# ['샷', '설비ID', '실린더압력', '주조압력',
#  '사이클타임', '비스킷두께', '형체력', '품질등급']
```

열 번호가 헷갈리면 columns 를 먼저 찍어 순서를 눈으로 확인할 것

posco

실습 3. 행과 열을 함께 잘라내기

loc 과 iloc 으로 같은 영역을 각각 지정해 결과가 같은지 확인

목표

같은 사각형 영역을 이름으로도 번호로도 지정해 두 결과가 일치하는지 확인

단계

① 연습용 30줄짜리 다이캐스팅 파일을 읽어 변수에 담기

② 칸 이름 목록을 출력해 각 칸이 몇 번째인지 눈으로 세어 보기

③ 앞 다섯 줄의 실린더압력과 형체력을 이름으로 지정해 뽑고 크기를 출력

④ 같은 영역을 번호만 써서 다시 뽑고 크기가 같은지 확인

예상 결과

두 결과 모두 5줄 2칸 · 끝을 세는 방식이 달라 적는 숫자는 하나 차이

실습 3 결과 해석

두 길이 같은 곳에 닿았는지 세 가지로 확인

원리

쉼표 앞은 행,
뒤는 열

loc 과 iloc 은 쉼표 구조가 완전히 같다.

다른 것은 그 자리에 이름을 적느냐 번호를 적느냐뿐이다.

함정

iloc 에서 끝 번호를
그대로 옮겨 적음

loc[0:4] 를 iloc[0:4] 로 옮기면
한 줄이 모자란다.

열도 [2, 6] 대신 2:7 로 적으면 사이 열이 전부 딸려 온다.

검산

shape 가 (5, 2)

두 결과의 shape 가 모두
(5, 2) 로 나오면 맞다.

(4, 2) 가 나왔다면 iloc 의 끝 번호를 하나 늘려야 한다.

posco

실습 3 정답

같은 사각형을 loc 과 iloc 으로 각각 지정

CODE

```
df = pd.read_csv('data/13_diecasting_small.csv')

print(list(df.columns))
# ['샷', '설비ID', '실린더압력', '주조압력',
#  '사이클타임', '비스킷두께', '형체력', '품질등급']

가 = df.loc[0:4, ['실린더압력', '형체력']]
print(가.shape)      # (5, 2)

나 = df.iloc[0:5, [2, 6]]
print(나.shape)      # (5, 2)
print(가.equals(나))  # True
```

마지막 줄의 equals 로 두 길이 같은 곳에 닿았는지 한 번에 확인

posco

loc과 iloc 슬라이싱 끝 포함 차이

오늘 수업에서 가장 실수가 잦은 부분 — 집중

01 · loc

`df.loc[0:2]`

0, 1, 2 → 세 줄

끝 라벨까지 포함 — 부른 이름은 다 옴

02 · iloc

`df.iloc[0:2]`

0, 1 → 두 줄

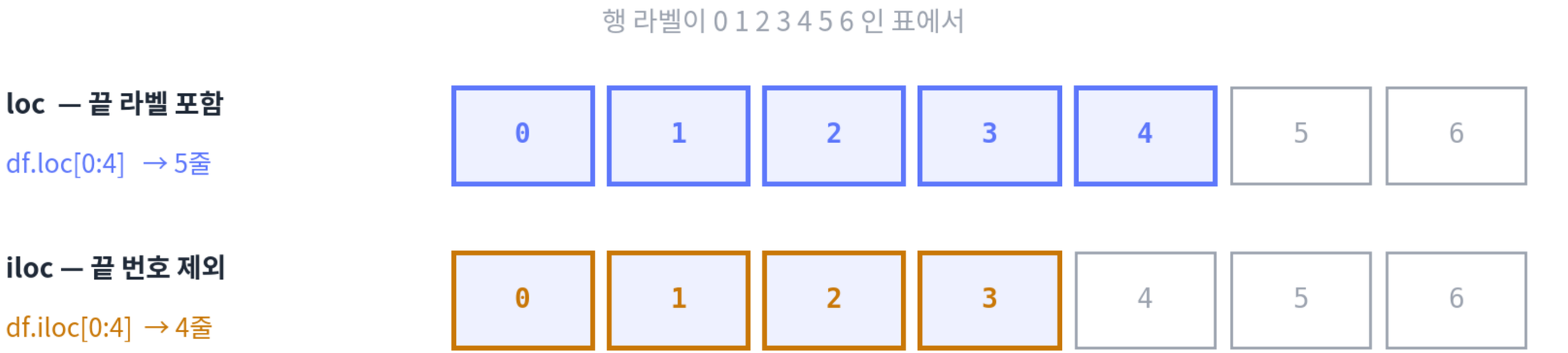
끝 미포함 — 파이썬 기본 슬라이싱 규칙

같은 0:2를 적었는데 줄 수가 다름 — 시험에 나오면 여기서 나옴

posco

loc 은 끝을 포함하고 iloc 은 포함하지 않는다

같은 0:4 를 적어도 줄 수가 다르다 — 이 단원에서 실수가 가장 잦은 자리



loc 은 이름표를 부르는 것이라 마지막 이름까지 부르고, iloc 은 개수를 세는 것이라 네 개에서 멈춘다

이름을 부르면 그 사람까지 오고, 개수를 세면 그 앞에서 멈춘다.

posco

03

종합 실습

열 선택 · loc · iloc 을 한 번에 엮어 필요한 구간만 뽑아내기

posco

실습 4. 필요한 구간만 뽑아내기

200행짜리 실제 로그에서 원하는 샷과 지표만 골라 표로 남기기

목표

앞쪽 10줄에서 설비 이름과 비스킷두께, 품질등급만 남기고 맨 뒤 5줄도 확인

단계

① 200줄짜리 다이캐스팅 로그를 읽어 전체 크기를 확인

② 칸 이름 목록을 출력해 칸 순서를 확인

③ 앞쪽 10줄에서 설비 이름과 비스킷두께, 품질등급 세 칸만 이름으로 골라내기

④ 맨 뒤 5줄 전체를 번호로 뽑아 앞 결과와 나란히 보기

예상 결과

전체는 200줄 8칸 · 골라낸 표는 10줄 3칸

실습 4 결과 해석

오늘 배운 세 가지가 이 한 줄 안에 다 들어 있다

원리

고르는 방법은 두 가지뿐

이름으로 부르는 길과 순서로 세는 길.

행이든 열이든, 표가 30행이든 200행이든 이 두 가지 안에서 해결된다.

함정

칸이 여덟 개인데 번호를 일곱으로 셈

설비ID 가 새로 생긴 열이라 번호가 하나씩 밀린다.

번호로 자를 때 columns 를 안 찍으면 매번 겪는 일이다.

검산

(10, 3) 과 (200, 8)

loc 결과가 (10, 3),
전체가 (200, 8) 이면 맞다.

(11, 3) 이 나왔다면 loc 의 끝 라벨 포함을 잊은 것이다.

실습 4 정답

loc 과 iloc 을 실제 로그에 그대로 적용

CODE

```
df = pd.read_csv('data/13_diecasting_shot.csv')
print(df.shape)              # (200, 8)
print(list(df.columns))
# ['샷', '설비ID', '실린더압력', '주조압력',
#  '사이클타임', '비스킷두께', '형체력', '품질등급']

앞열샷 = df.loc[0:9, ['설비ID', '비스킷두께', '품질등급']]
print(앞열샷.shape)          # (10, 3)

뒤다섯 = df.iloc[-5:]
print(뒤다섯.shape)          # (5, 8)
print(뒤다섯['샷'].tolist()) # [196, 197, 198, 199, 200]
```

loc 은 0:9 로 열 줄 · iloc 의 음수는 뒤에서부터 세는 표기

posco

행·열 선택 핵심 정리

오늘 단 하나만 가져간다면 — 이름표로 고를 땐 loc, 순서로 고를 땐 iloc

01 · loc

라벨로 선택

끝 라벨까지 포함 · 조건 필터 가능

이름표로 부르니까 부른 이름까지 다 옴 — 조건 필터의 무대

정렬 뒤에는 옛 이름표를 가리키므로 reset_index 로 다시 붙인다

02 · iloc

위치로 선택

끝 직전까지 · 정수만

파이썬 기본 슬라이싱 규칙 — 정확히 N개가 필요할 때

다음 단원 — 비스킷두께 16 이상 같은 조건으로 고르기, 이상 샷을 찾는 가장 강력한 무기